

Praktiskā kombinatoriālā optimizācija – mazais praktiskais darbs

Koda un rezultātu repozitorijs: <https://github.com/Paddlebear/KillerSudokuSolver>

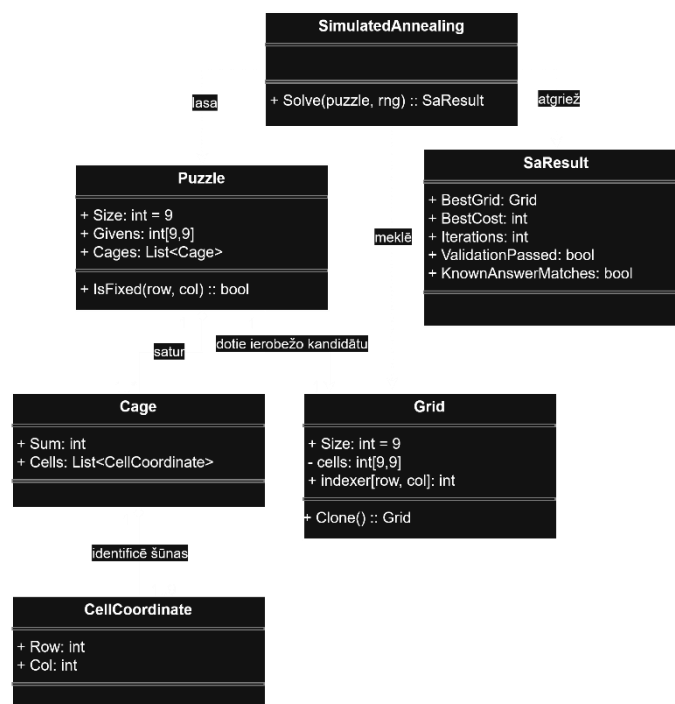
1) Uzdevuma formulējums

Praktiskais darbs apskata Killer Sudoku risināšanu. Killer Sudoku ir sudoku variants, kurā režģis sastāv no parastajiem sudoku ierobežojumiem (9x9 režģis, sadalīts deviņās 3x3 kastēs, kas jāaizpilda ar cipariem 1-9, un rindās, kolonnās un kastēs nedrīkst būt dublikāti), bet režģī ir arī aprakstīti neregulāru formu būri, kas sastāv no vismaz vienas šūnas. Režģis ir aizpildīts ar šiem būriem, un ir dotas būra robežas un tajā iekļauto šūnu summa. Klasiski šo variantu spēlē bez dotām šūnām, bet ir arī iespējams spēlēt ar noteiktu daudzumu dotajiem cipariem, kuru skaits ir parasti mazāks nekā parastā sudoku.

Risinātājs meklē aizpildītu režģi, kas izpilda visus šos noteikumus. Algoritms apskata visus kandidātu režģus no dotās pozīcijas, novērtē katru pēc pārkāpumu skaita un svara, un izvēlas optimālo režģi kā nākamo gājieni. Mērķis ir režģa novērtējums jeb izmaksa = 0; ja tas ir sasniegts, tad iegūto režģi neatkarīgi validē un atgriež kā risinājumu.

2) Algoritma apraksts – *Simulated Annealing*

a) Domēns

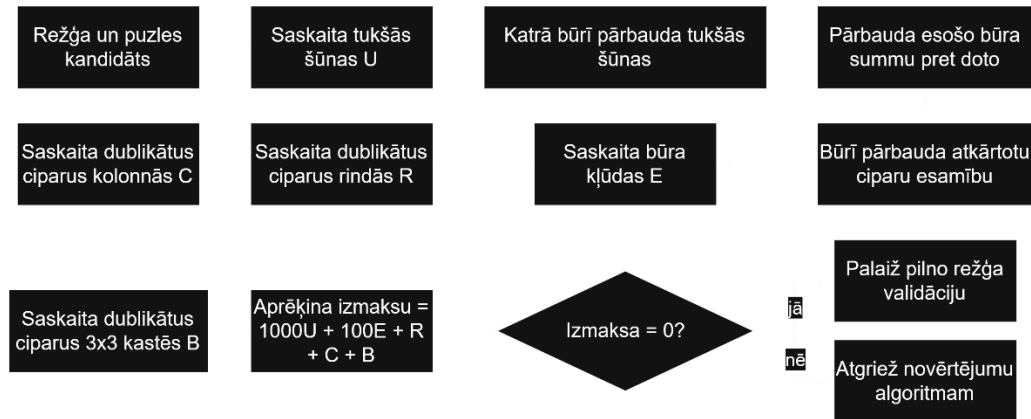


Režģis aprakstīts kā klase Grid, kas sastāv no 9x9 veselu skaitļu masīva, kas var saturēt ciparus 1-9. Tam ir funkcija Clone(), kas ļauj klonēt režģa pašreizējo konfigurāciju. Būris aprakstīts kā klase Cage, kas satur tā summu (dota) un šūnas kā sarakstu ar koordinātām. Mīkla aprakstīta kā klase Puzzle, kas sastāv no doto šūnu vērtībām kā 9x9 masīvu analogiski Grid, un sarakstu ar mīklas būriem. Dotās vērtības nevar tikt mainītas.

Algoritms atgriež rezultātu, kas aprakstīts klasē SaResult, kas satur labāko (ar mazāko izmaksu) režģi, mazāko izmaksu, iterāciju skaitu, vai rezultāts ir veiksmīgi validēts, un vai iegūtais rezultāts atbilst dotajam parauga rezultātam. Vērts minēt, ka Killer Sudoku mīklai bez

dotajām šūnām var būt vairāk par vienu risinājumu, tāpēc fakts, ka iegūtais rezultāts nesakrīt ar doto, nenozīmē, ka tas nav derīgs, ja tas ir veiksmīgi izgājis validāciju.

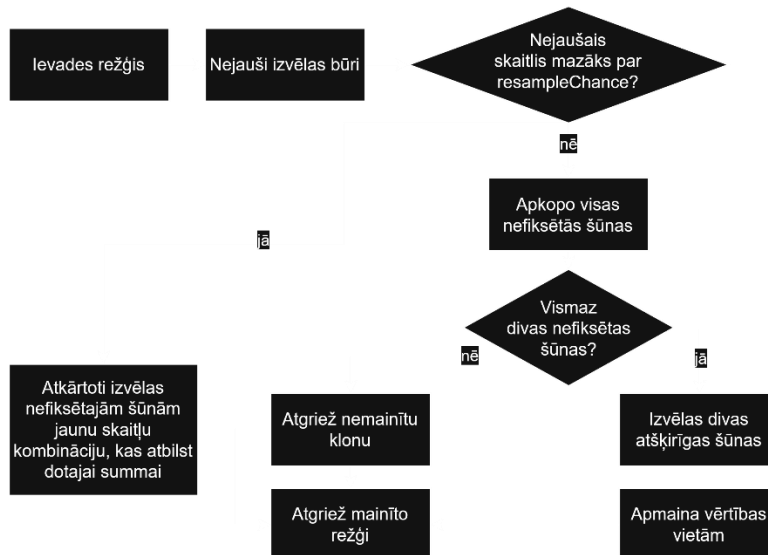
b) Novērtējums



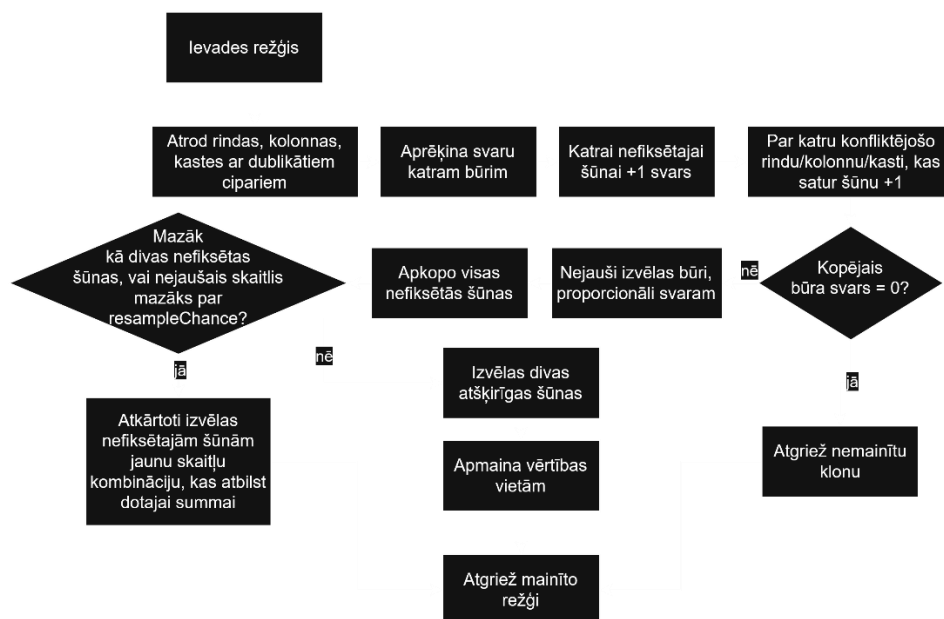
Kandidāta režģa izmaksa tiek aprēķināta pēc formulas $1000U + 100E + R + C + B$, kur U ir neaizpildītās šūnas, E ir būra kļūdas (nesakrītoša summa, būris ar tukšu šūnu, vai dublikāti cipari), un R, C, B ir attiecīgi rindu, kolonnu, un kastu konflikti (dublikāti cipari). Algoritma mērķis ir atrast režģi, kuram izmaksa ir 0. Vērtības ir svērtas tā, lai nekad neakceptētu režģi ar tikai tukšām vērtībām, kā arī lai smagāk svērtu būru kļūdas, jo algoritma izstrādes gaitā tika novērots, ka biežākais iemesls mīklas neatrisināšanai ir tieši būra kļūdas.

c) Gājiens

Tika izveidotas divas dažādas pieejas.



Pirmā pieeja (MoveGenerator jeb MG1) nejauši izvēlas kādu no mīklas būriem. Gājienam dota 15% iespēja atkārtoti izvēlēties būra ciparu kombināciju, kuras summa ir dotā būra summa, atstājot dotās šūnas nemainītas. Citādi algoritms apmaina divu nefiksēto šūnu vērtības, ja tādas šūnas ir pieejamas.



Otrā pieeja (MoveGenerator2 jeb MG2) identificē rindu, kolonnu un kastu konfliktus, un piešķir svaru mīklas būriem balstoties uz to, vai būris satur kopīgas šūnas ar konfliktējošajām rindām/kolonnām/kastēm. Proporcionāli svaram nejauši izvēlas būri, un pielieto to pašu gājienu, kā pirmajā pieejā.

Abos gadījumos tiek klonēts esošais režģis salīdzināšanai, lai pārbaudītu, vai gājiens tiešām ir atrisinājis konfliktu.

d) Algoritms

Risinātājs izmanto *simulated annealing* pieeju. Sākotnēji režģa būri tiek aizpildīti ar ciparu kombinācijām, kas atbilst būra dotajai summai, ņemot vērā dotās šūnas, ja tādas ir. Ar šādu metodi var uzreiz samazināt būra konfliktu skaitu, bet visticamāk būs rindu/kolonnu/kastu konflikti. Šim režģim aprēķina tā izmaksu, saglabā kā gan pašreizējo, gan labāko kandidātu. Tālāk gājiena algoritms piedāvā jaunu režģi, kas izveidots no pašreizējā. Ja kandidāta izmaksa ir mazāka, to pieņem; ja lielāka, tad pieņem ar varbūtību $e\left(-\frac{\Delta}{T}\right)$, kur Δ ir kaimiņa izmaksa - pašreizējā izmaksa, un T ir temperatūra – jo lielāka temperatūra, jo lielāka iespēja, ka algoritms pieņems sliktāku kandidātu, kas palīdz izkļūt no lokālajiem minimumiem.

Pēc katras iterācijas temperatūra tiek samazināta, reizinot to ar dzesēšanas ātrumu. Ja meklēšana ir stagnējusi (nav atrasts labāks kandidāts) noteiktu skaitu iterāciju un temperatūra ir kritusi zem noteiktas robežas, meklēšana tiek atsākta no labākā kandidāta, tiek izpildīti trīs nejauši “grūdiens” gājieni un temperatūra tiek pacelta līdz konfigurētai vērtībai. Algoritms turpina darbu, līdz tas atrod režģi ar izmaksu 0, kas ir izgājis validāciju, vai arī ir sasniegts uzskarsēšanas reižu vai iterāciju skaita limits.

Algoritma izstrādes gaitā tika izmēģinātas vairākas pieejas. Pirmā iterācija koncentrējās uz rindu/kolonnu/kastu konfliktiem un gājienu izpildi tikai šo struktūru ietvaros, kas palielināja algoritma izpildes laiku un noveda pie zemiem atrisināšanas rādītājiem, jo bija būru summu konflikti. Otrā iterācija mēģināja problēmu atrisināt, ieviešot pilnīgi nejaušus gājienu, bet algoritms bija vēl lēnāks un sliktāks. Trešā iterācija tika konstruēta, koncentrējoties uz būros balstītu inicializāciju un gājieniem. Ceturtā iterācija (darbā aprakstītais algoritms) gājienu ziņā ir identisks, bet tajā tika uzlabota izmaksu funkcija un pievienota neatkarīga režģa validācija.

3) Testēšanas apraksts

a) Darbā aprakstītā algoritma testēšana

Mīklu piemēri tika ņemti no <https://github.com/UvA-KR16/KilerSudoku>. Repozitorijā ir 12000 piemēri, sadalīti pa maksimālo būra izmēru. No katra būra izmēra mapes tika nejauši izvēlētas trīs mīklas un to risinājumi pārbaudei – kopā 24 mīklas. Katras mīklas risināšana tika testēta ar 10 sēklu vērtībām (nejaušajiem skaitļiem) un diviem dzesēšanas ātrumiem 0,9999 un 0,99995 – kopā katrai mīklai 20 testi, kopumā 480 testi katrai gājienu pieejai. Testā tika dots maksimālais iterāciju skaits 2500000, stagnēšanas iterāciju limits 20000, minimālā temperatūra 0,01, maksimālais uzskarsēšanu skaits 25, sākuma temperatūra 2,0 un uzskarsēšanas temperatūra 1,0.

Testu rezultāti:

Maksimālais būra izmērs	Atrisinātās un validētās mīklas (vidēji)		Vidējais laiks (milisekundes)		Vidējā labākā izmaksa		Vidējā labākā izmaksa (neatrisinātās)		Vidējais laiks (milisekundes, atrisinātās)	
	MG1	MG2	MG1	MG2	MG1	MG2	MG1	MG2	MG1	MG2
2	98.3333%	100.0000%	527.5667	668.4000	0.0667	0.0000	4.0000	–	456.2542	668.4000
3	100.0000%	100.0000%	957.8167	1040.3000	0.0000	0.0000	–	–	957.8167	1040.3000
4	43.3333%	58.3333%	4316.5500	5912.7000	4.3667	3.1833	7.7059	7.6400	2077.3077	3192.5143
5	23.3333%	56.6667%	5558.0000	7191.6667	3.5000	2.0167	4.5652	4.6538	3437.2857	5315.4706
6	20.0000%	38.3333%	5532.6333	7765.8000	4.1500	2.3500	5.1875	3.8108	3045.7500	4125.4783
7	33.3333%	35.0000%	5144.3333	7910.7667	3.9833	3.0167	5.9750	4.6410	2907.1000	4837.1905
8	45.0000%	50.0000%	4608.9333	7399.1833	2.5000	1.6833	4.5455	3.3667	2811.9630	5468.2000
9	68.3333%	73.3333%	2353.4333	3763.3333	0.9000	1.2833	2.8421	4.8125	678.4390	1687.6818
Kopā	53.9583%	63.9583%	3624.9083	5206.5188	2.4333	1.6917	5.2851	4.6936	1486.2934	2702.7915

Pa dzesēšanas ātrumiem un mīklām:

CoolingRate 0.9999									
Column Labels		Atrisin. Vidēji		Vidējais laiks, milisekundes		Total Atrisin. Vidēji		Total Vidējais laiks, milisekundes	
Row Labels	MG1	MG2	MG1	MG2					
2/1263.killer	1	1	96	112.2	1	104.1			
2/560.killer	0.9	1	1133.4	1530.4	0.95	1331.9			
2/711.killer	1	1	648.1	763.5	1	705.8			
3/1263.killer	1	1	655.8	641.2	1	648.5			
3/413.killer	1	1	1519.5	1198.9	1	1359.2			
3/519.killer	1	1	416.3	1053.4	1	734.85			
4/1374.killer	0	0.1	4264.3	6097.9	0.05	5181.1			
4/1434.killer	0.9	0.9	1325.5	2009.8	0.9	1667.65			
4/350.killer	0.1	0.6	4141.2	4598.7	0.35	4369.95			
5/1118.killer	0	0.4	4102.4	5378.5	0.2	4740.45			
5/1439.killer	0.5	0.8	3030.8	4242.8	0.65	3636.8			
5/186.killer	0.2	0.4	3769.2	5492.7	0.3	4630.95			
6/1320.killer	0.1	0.3	3848.9	5851.3	0.2	4850.1			
6/146.killer	0.5	0.7	3252.4	5085.5	0.6	4168.95			
6/748.killer	0.1	0.3	3804.3	5191.4	0.2	4497.85			
7/461.killer	0	0	3959.3	6379.1	0	5169.2			
7/654.killer	0.8	0.5	2319.7	4594.2	0.65	3456.95			
7/965.killer	0.5	0.5	3237.1	5578	0.5	4407.55			
8/1084.killer	0.3	0.3	3668.6	5245.6	0.3	4457.1			
8/194.killer	0.6	0.5	2517.2	5156.1	0.55	3836.65			
8/348.killer	0.6	0.6	2266.8	4688.8	0.6	3477.8			
9/13.killer	1	1	1058.6	1196.4	1	1127.5			
9/602.killer	1	1	376.2	1210.6	1	793.4			
9/996.killer	0.1	0.2	3772.1	6055.1	0.15	4913.6			
Kopā	55.0000%	62.9167%	2465.9875	3723.004187	58.9583%	3094.495833			

CoolingRate	0.99995						
Column Labels							
Atrisin. Vidēji		Vidējais laiks, milisekundes		Total Atrisin. Vidēji		Total Vidējais laiks, milisekundes	
Row Labels	MG1	MG2	MG1	MG2			
2/1263.killer	1	1	134.1	110.7	1		122.4
2/560.killer	1	1	1037.3	986.4	1		1011.85
2/711.killer	1	1	116.5	507.2	1		311.85
3/1263.killer	1	1	861	782.2	1		821.6
3/413.killer	1	1	1285.2	1498.8	1		1392
3/519.killer	1	1	1009.1	1067.3	1		1038.2
4/1374.killer	0	0.2	8392.2	12037.5	0.1		10214.85
4/1434.killer	1	1	1757.3	1183.7	1		1470.5
4/350.killer	0.6	0.7	6018.8	9548.6	0.65		7783.7
5/1118.killer	0	0.2	8111.2	12451.9	0.1		10281.55
5/1439.killer	0.5	0.8	7036.7	8340.1	0.65		7688.4
5/186.killer	0.2	0.8	7297.7	7244	0.5		7270.85
6/1320.killer	0.1	0.4	7492.1	9480.8	0.25		8486.45
6/146.killer	0.3	0.3	7394.7	10674	0.3		9034.35
6/748.killer	0.1	0.3	7403.4	10311.8	0.2		8857.6
7/461.killer	0	0.2	7895.9	11342.1	0.1		9619
7/654.killer	0.4	0.6	6645	8899	0.5		7772
7/965.killer	0.3	0.3	6809	10672.2	0.3		8740.6
8/1084.killer	0	0.5	7829.9	10558	0.25		9193.95
8/194.killer	0.3	0.6	6684	9098.4	0.45		7891.2
8/348.killer	0.9	0.5	4687.1	9648.2	0.7		7167.65
9/13.killer	1	1	654.1	1076.6	1		865.35
9/602.killer	1	1	446.9	1087.8	1		767.35
9/996.killer	0	0.2	7812.7	11953.5	0.1		9883.1
Kopā	52.9167%	65.0000%	4783.829167	6690.033333	58.9583%		5736.93125

Kopumā MG2 tika galā ar gandrīz 64% mīklu, bet MG1 atrisināja 54%. MG2 arī bija labāka vidējā izmaksa 1,69, bet MG1 – 2,433. Daļa no tā ir skaidrojama ar faktu, ka kopumā MG2 atrisināja vairāk mīklu, bet, ja salīdzina neatrisināto mīklu vidējo labāko izmaksu, tad var redzēt, ka tā arī ir mazāka MG2 – tāpēc MG2 kopā tuvāk pietuvojās derīgam risinājumam. Līdzīgi var salīdzināt vidējo laiku tikai tiem testiem, kas tika veiksmīgi atrisināti, un šeit var redzēt, ka MG2 ir aptuveni 1,8x lēnāks par MG1.

Rezultātos arī parādās, ka mīklās ar maks. būra izmēru 2 un 3 mīklai tika atrasts derīgs risinājums gandrīz vienmēr, bet, jo lielāks būris, jo sliktāk algoritmam gāja, līdz būra izmēram 6, un tad rezultāti uzlabojās. Šis var liecināt par likumsakarību, ka patiešām mīklas ar maks. būra izmēru 5-6 ir grūtāk atrisināmas, nekā 8-9, bet tas var arī liecināt par labu/sliktu veiksmi mīklu piemēru izvēlē, kas tika veikts nejauši.

Dzesēšanas ātrumam ietekme uz algoritma mīklu atrisināšanas metriku nebija liela, bet interesanti ir tas, ka ietekme atšķīrās, atkarībā no algoritma. MG1 precizitāte samazinājās, MG2 – palielinājās. Daudz lielāka ietekme bija uz vidējo risinājuma laiku. Mīklām, kas tika atrisinātas tuvu 100% testos dzesēšanas ietekme uz ātrumu nebija liela, kas pasaka, ka kopumā, ja mīkla tiks atrisināta, tad tas notiks noteiktu iterāciju skaitā. Ja mīkla netiek atrisināta, tad algoritms apstājas pēc noteikta laika – šajā gadījumā ar dotajiem testa parametriem noteicošais faktors bija uzskarsēšanas reižu skaits. Neatrisināta mīkla pie dzesēšanas ātruma 0,9999 apstājās pēc ~1,2milj. iterācijām, bet 0,99995 – pēc ~2,4milj, jeb gandrīz precīzi 2x vairāk, kas arī noved pie tā, ka neatrisinātam gadījumam tiek patērēts 2x vairāk laiks ar ātrumu 0,99995. Pēdējie testi tika veikti ar ļoti jaudīgu procesoru (Intel Core Ultra 5 250KF Plus, 4,2GHz), kas nozīmēja, ka MG1 neatrisināta mīkla prasīja ~4 un ~8 sekundes respektīvi dzesēšanas ātrumiem, un MG2 – respektīvi ~6,6 un ~13 sekundes. Testi tika arī iepriekš laisti uz mazāk jaudīga portatīvā datora, kur patērētais laiks varēja būt līdz pat 4x lielāks.

Rezultāti tika arī salīdzināti ar dotu atbildi, un dotā atbilde sakrita aptuveni 40% atrisināto gadījumu (kopā visos testos MG1: 20%, MG2: 26%), bet tas nenozīmē, ka atrastais režģis nav derīgs – Killer Sudoku mīklai ir vairāki iespējami risinājumi, kā iepriekš minēts.

No šī visa var secināt, ka MG2 ir precīzāks, bet arī lēnāks papildus aprēķinu un būru svēršanas dēļ.

b) Saistītie pētījumi

Klasiskā sudoku risināšanas problēma ir plaši apskatīta literatūrā, bet ir daudz mazāk informācijas par sudoku variantiem, tāpēc ar šiem pētījumiem nav iespējams tieši salīdzināt, jo Killer Sudoku būru ierobežojumi (un tādējādi arī doto šūnu trūkums) problēmu sarežģī.

Stohastisko metožu pielietošana sudoku risināšanai ir aprakstīta literatūrā¹, tomēr tā nav bieži izmantota pieeja, jo, tā kā “optimālais” (jeb pareizais) risinājums ir ar izmaksu precīzi 0, un lielāka izmaksa nav akceptējama, problēma un risinājums parasti tiek formulēta un veikta ar deterministiskām pieejām, piemēram, *backtracking* vai CSP (*constraint satisfaction problem*); šim tipiski pielieto SAT-algoritmus.

Tika apskatīti divi projekti, kas apraksta Killer Sudoku risināšanas problēmu. Viens no tiem arī izmanto *simulated annealing* (<https://github.com/2pa4ul2/Killer-Sudoku>, autors – Paul Adrian Torres no Filipīnu Tehnoloģiskās universitātes), bet pēc koda izpētes tika secināts, ka implementācija realitātē neatbilst SA principiem, jo risinājuma izmaksa nekad nevar palielināties (izmaksu funkcija tikai skatās uz to, vai šūna ir aizpildīta), risinājums atrisina tikai 4x4 režģi, un meklēšana tiek pilnībā restartēta pēc aptuveni 110 iterācijām ($0.9^{110} \approx 0.00001$), kas nebūtu pietiekami 9x9 režģim.

Otrs projekts (<https://github.com/UvA-KR16/KillerSudoku>, autori – Shuai Wang un Aashish Venkatesh) izmanto SAT-algoritmu. No šī projekta arī tika ņemti testa dati (.killer un .ans faili). Projekts ir rakstīts Python 2 (palaišanai tika konvertēts uz Python 3) un izmanto Pycosat bibliotēku, sākotnēji pārvēršot doto .killer mīklu par CNF formulu, kas tālāk tiek nodots Pycosat apstrādei.

Programma tika palaista WSL ar tiem pašiem testa piemēriem, kas tika izmantoti šajā darbā.

```
the solver is: eval.py
evaluating 2
total time for cage size 2 is = 0.5653, among which, the solving time is 0.0074
they have an avg. no. arith clauses of 1494.0 and a total of 8946.0
evaluating 3
total time for cage size 3 is = 0.5724, among which, the solving time is 0.0078
they have an avg. no. arith clauses of 1541.6666666666667 and a total of 8993.666666666666
evaluating 4
total time for cage size 4 is = 0.5742, among which, the solving time is 0.0105
they have an avg. no. arith clauses of 1699.0 and a total of 9151.0
evaluating 5
total time for cage size 5 is = 0.5635, among which, the solving time is 0.0108
they have an avg. no. arith clauses of 1630.0 and a total of 9082.0
evaluating 6
total time for cage size 6 is = 0.5853, among which, the solving time is 0.0095
they have an avg. no. arith clauses of 1550.6666666666667 and a total of 9002.666666666666
evaluating 7
total time for cage size 7 is = 0.5886, among which, the solving time is 0.0098
they have an avg. no. arith clauses of 1369.6666666666667 and a total of 8821.666666666666
evaluating 8
total time for cage size 8 is = 0.5895, among which, the solving time is 0.0099
they have an avg. no. arith clauses of 1323.0 and a total of 8775.0
evaluating 9
total time for cage size 9 is = 0.5754, among which, the solving time is 0.0106
they have an avg. no. arith clauses of 1212.0 and a total of 8664.0
```

Pats SAT-algoritms atrod pareizu atbildi gandrīz uzreiz (*solving time*, laiks attēlots sekundēs uz piemēru). Papildus procesi palielina prasīto laiku līdz līmenim, kas salīdzināms ar darbā aprakstīto un testēto algoritmu, bet SAT pieeja joprojām ir ātrāka (ja salīdzina *total time* ar iepriekš aprakstīto vidējo laiku atrisinātiem testiem), kas arī atbilst idejai, ka problēmai, kurai ir pareizs (ne tikai optimāls) risinājums, labāk ir izmantot deterministiskas pieejas.

¹ https://www.researchgate.net/publication/220403361_Metaheuristics_can_solve_Sudoku_puzzles